CLOUD OF THINGS · CAHIER DES CHARGES

Smart Museum — Architecture technique et contraintes

Plateforme IoT intelligente et sécurisée pour la conservation préventive du patrimoine culturel

Sommaire

1. Objectif du bloc technique
2. Architecture globale
3. Choix technologiques par couche
4. Flux de données et contrat d'interface
5. Diagramme UML de déploiement
6. Contraintes de temps, budget et périmètre
7. Contraintes technologiques et de sécurité
8. Méthodologie XP et intégration
9. Annexe A — Source PlantUML du déploiement

1. Objectif du bloc technique

Ce bloc explique comment Smart Museum sera construit et déployé : architecture globale, diagramme UML de déploiement et contraintes du projet. Il s'appuie sur la référence partagée du projet et sur les exigences du cours Cloud of Things : Jakarta EE 11, Node-RED & MQTT, méthode XP et déploiement Cloud IaaS depuis GitHub.

Il couvre trois livrables : les sections techniques du cahier des charges, le diagramme de déploiement et les contraintes qui encadrent la réalisation.

2. Architecture globale

Smart Museum est organisé en couches : les capteurs mesurent, la passerelle Raspberry Pi publie en MQTT, Node-RED intègre les flux, Jakarta EE applique les règles et sert les deux PWA. MongoDB et le modèle Llama ne sont joignables que par le backend.

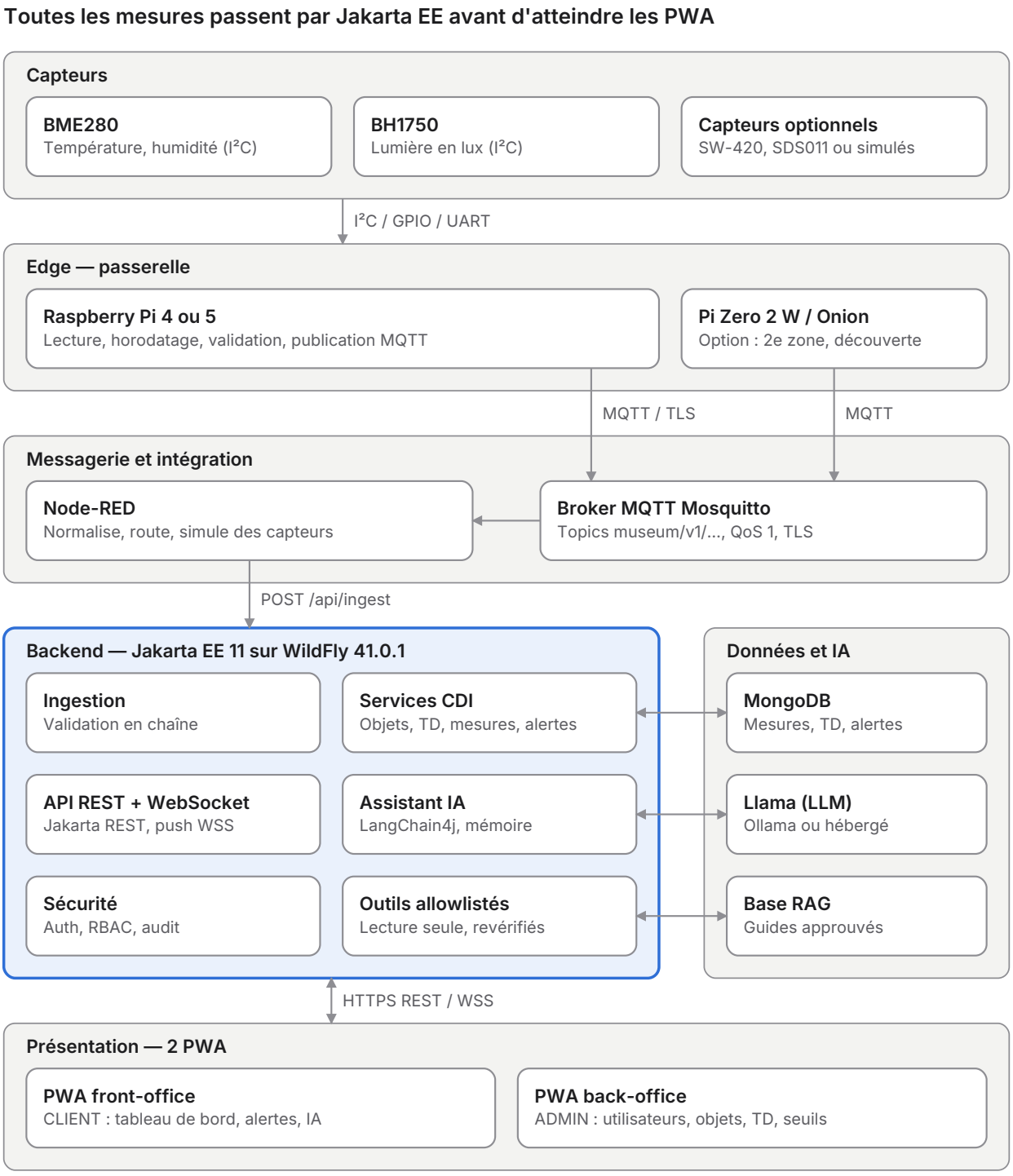


Figure 1 — Architecture globale : 5 couches, du capteur aux PWA. Une mesure descend de haut en bas ; les flèches doubles sont des échanges requête/réponse.

2.1 Rôle de chaque couche

- **Capteurs** : BME280 et BH1750 reliés en I²C ; capteurs supplémentaires réels ou simulés, toujours étiquetés comme tels.
- **Edge** : la passerelle identifie la source, horodate, valide puis publie ; un 2e nœud (Pi Zero 2 W / Onion) est optionnel.
- **Messagerie et intégration** : le broker découple émetteurs et consommateurs ; Node-RED normalise les messages et les transmet au backend.
- **Backend Jakarta EE** : seul propriétaire des règles métier, de la sécurité, des alertes et de l'orchestration IA.
- **Données et IA** : MongoDB pour la persistance ; Llama et la base RAG appelés via LangChain4j et des outils contrôlés.
- **Présentation** : deux PWA par rôle (CLIENT et ADMIN), utilisables sur PC, tablette ou smartphone.

2.2 Patrons de conception

Patron	Application
Observer	Une nouvelle mesure ou alerte notifie les abonnés WebSocket et les services intéressés.
Chain of Responsibility	Validation à l'ingestion : schéma → identité de l'objet → plage/qualité → confiance → persistance.
Strategy	Choix de la règle de seuil ou du parseur selon le type de capteur.
Event Aggregation	Regroupement des alertes répétées en un seul incident.

3. Choix technologiques par couche

Chaque composant a une seule responsabilité ; aucune logique métier n'est dupliquée entre Node-RED et Jakarta EE.

Couche	Composant	Technologie	Rôle unique	Justification
Capteurs	Capteurs environnementaux	BME280 (température, humidité, pression), BH1750 (lumière) ; SW-420, SDS011 en option	Mesurer les conditions physiques	Minimum 1 capteur réel ; bus I²C 3,3 V compatible Raspberry Pi
Edge	Passerelle principale	Raspberry Pi 4 ou 5, Raspberry Pi OS, agent passerelle	Lire les capteurs, horodater, valider, publier en MQTT, exposer sa santé	Matériel de référence ; bufferisation locale en option (P2)
Edge (option)	Nœud secondaire	Raspberry Pi Zero 2 W ou Onion	Deuxième zone / démonstration de découverte	Non requis pour le P0
Messagerie	Broker MQTT	Eclipse Mosquitto	Découpler émetteurs et consommateurs (topics, QoS)	MQTT imposé par le cours ; clients authentifiés, TLS en déploiement

Couche	Composant	Technologie	Rôle unique	Justification
Intégration	Flux IoT	Node-RED	Normaliser, router, observer les messages ; simuler des capteurs étiquetés « simulated »	Outil nommé par le cours ; Node-RED ≠ Node.js
Backend	Application métier	Jakarta EE 11 sur WildFly 41.0.1.Final (Java 21)	API, règles, alertes, sécurité, orchestration IA	Plateforme imposée par le cours
Backend	API	Jakarta REST (JSON), Jakarta WebSocket, CDI	Opérations CRUD et push temps réel	Standards Jakarta EE 11
Données	Base de données	MongoDB et driver Java officiel	Persister utilisateurs, objets, TD, mesures, alertes, documents	Documents JSON proches des TD JSON-LD ; driver vérifié par un prototype CRUD
Sémantique	Description des objets	W3C WoT Thing Description 1.1 en JSON-LD 1.1 ; RFC 8288 pour les liens	Décrire propriétés, actions, événements et formes d'interaction	Standards exigés
IA	Assistant	Llama (Ollama ou endpoint hébergé), LangChain4j, RAG, outils allowlistés, MCP en option	Aide à la décision fondée sur les mesures autorisées et les documents approuvés	Pas d'inférence LLM sur le Raspberry Pi
Présentation	Deux PWA	Front-office (CLIENT) et back-office (ADMIN), un framework commun	Interfaces par rôle, installables sur mobile	Exigé par le sujet
Infrastructure	Hébergement	VM Cloud IaaS Linux, reverse proxy HTTPS (Nginx), déploiement par git clone/pull	Exposer www / api / iam / test	Exigé par le cours

4. Flux de données et contrat d'interface

4.1 Chemin d'une mesure

Node-RED s'abonne aux topics MQTT, normalise le message puis l'envoie à un endpoint sécurisé de Jakarta EE. Jakarta EE revalide tout et reste seul maître des règles métier. Un message n'est traité qu'une fois grâce à l'identifiant `measurementId` et au contrôle de doublon.

1. Le capteur est lu par la passerelle via I²C ; la passerelle ajoute `deviceId`, `zoneId` et l'horodatage `observedAt`.
2. La passerelle publie en MQTT sur `museum/v1/{zoneId}/{deviceId}/telemetry` (QoS 1, TLS en déploiement).
3. Node-RED valide le schéma, renseigne `source` (real / simulated), puis appelle `POST /api/ingest/measurements` avec un jeton de service.
4. Jakarta EE applique la chaîne de validation : schéma → identité de l'objet → plage/qualité → confiance → persistance MongoDB (ajout de `receivedAt`).

5. Le moteur de règles compare aux seuils ; en cas de dépassement, une alerte est créée, avec protection contre les tempêtes d'alertes.
6. Le service WebSocket pousse mesure et alerte aux tableaux de bord autorisés ; l'historique passe par l'API REST.

4.2 Convention des topics MQTT

Topic	Contenu
`museum/v1/{zoneId}/{deviceId}/telemetry`	Mesures
`museum/v1/{zoneId}/{deviceId}/status`	Santé, online/offline (message LWT)
`museum/v1/{zoneId}/{deviceId}/event`	Événements (vibration, alarme locale)
`museum/v1/{zoneId}/{deviceId}/td`	Annonce de la Thing Description pour la découverte

4.3 Format d'une mesure et Thing Description

Une mesure contient `measurementId`, `deviceId`, `zoneId`, `property`, `value`, `unit`, `observedAt`, `receivedAt`, `source` et `quality`. La Thing Description est une ressource séparée en JSON-LD 1.1 (`@context`, `id`, `properties`, `actions`, `events`, `forms`).

4.4 Découverte d'un objet

Annonce MQTT sur `/td` → validation de l'identifiant et de la TD → enregistrement en statut `pending` → approbation ADMIN dans le back-office → statut `active`.

4.5 Interfaces vers les PWA

HTTPS REST (`/api/devices`, `/api/measurements`, `/api/alerts`, `/api/gateways`, `/api/ai/chat`) et WSS pour le temps réel. Le navigateur n'accède jamais directement à MongoDB ni au broker.

4.6 Chaîne de l'assistant IA

Question → authentification et périmètre (rôle, zones) → LangChain4j → Llama → outils en lecture seule (`getLatestMeasurements`, `getHistoricalMeasurements`, `getActiveAlerts`, `getDeviceStatus`, `searchApprovedGuidance`) revérifiés côté serveur → réponse distinguant faits mesurés et recommandations.

5. Diagramme UML de déploiement

Le système se déploie sur 6 nœuds physiques ou logiques ; une seule VM Cloud IaaS héberge le broker, Node-RED, WildFly, MongoDB et le serveur web des PWA. Seuls les ports 443 (HTTPS/WSS) et 8883 (MQTT/TLS) sont exposés à l'extérieur.

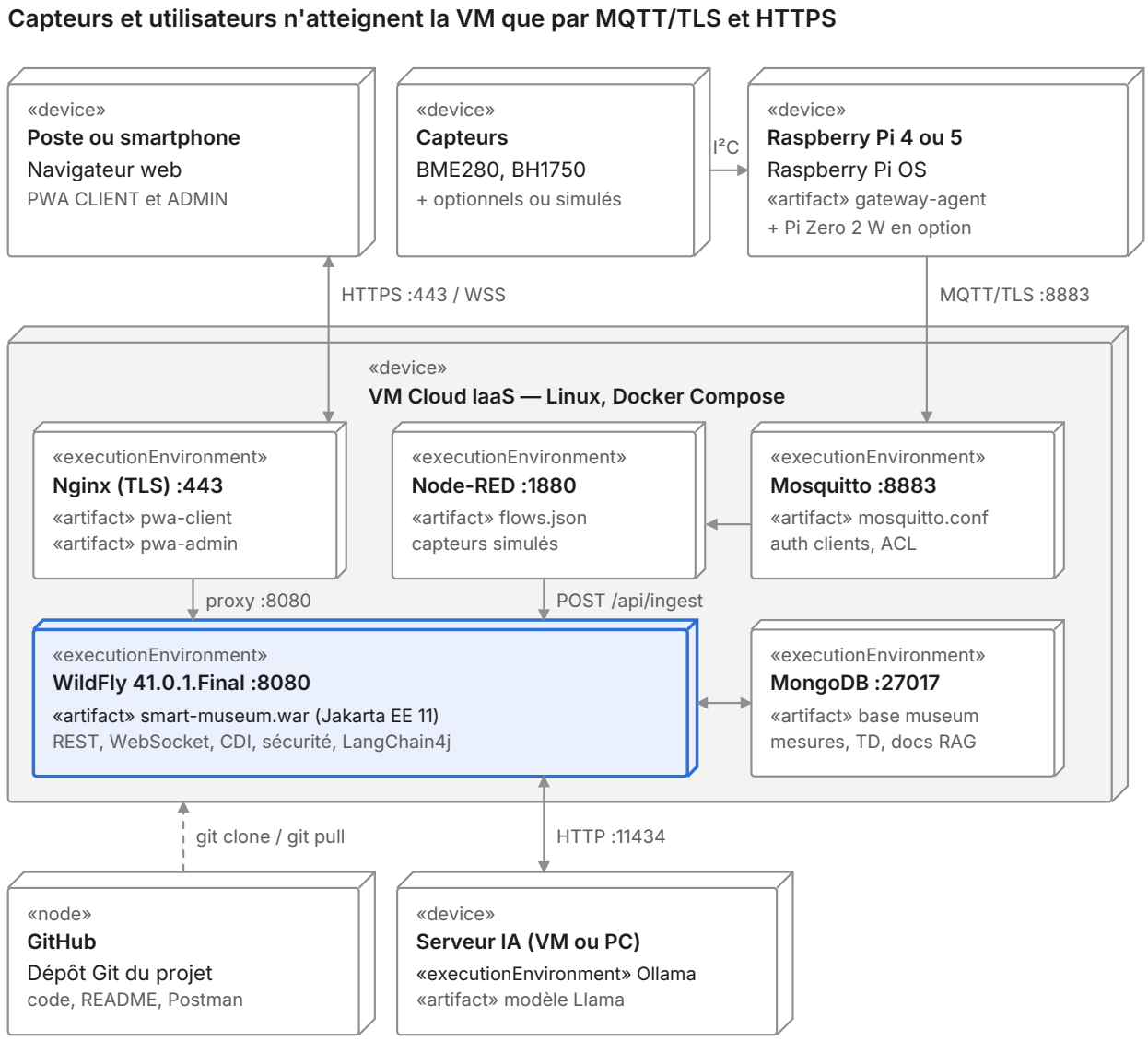


Figure 2 — Diagramme UML de déploiement : 6 nœuds, 5 environnements d'exécution.

Notation UML : cube = nœud (« device » ou « executionEnvironment ») ; « artifact » = élément déployé ; flèche = chemin de communication ; pointillé = dépendance de déploiement. Les ports indiqués sont les valeurs par défaut des outils.

5.1 Chemins de communication

Chemin	Protocole / port	Sécurité
Capteurs → Raspberry Pi	I²C (GPIO 3,3 V)	Liaison filaire locale
Raspberry Pi → Mosquitto	MQTT sur TLS, 8883	Identifiants et certificat par passerelle, ACL par topic
Mosquitto → Node-RED	MQTT, 1883	Réseau interne de la VM uniquement
Node-RED → WildFly	HTTP, `POST /api/ingest`	Réseau interne et jeton de service
Navigateur → Nginx	HTTPS 443, WSS	TLS, jeton d'authentification utilisateur
Nginx → WildFly	HTTP 8080 (reverse proxy)	Interne ; WildFly non exposé
WildFly → MongoDB	Driver Java, 27017	Interne, compte de base de données dédié
WildFly → Ollama	HTTP 11434	Interne, ou tunnel chiffré si le serveur IA est séparé
GitHub → VM	`git clone` / `git pull` (SSH ou HTTPS)	Clé de déploiement en lecture seule

6. Contraintes de temps, budget et périmètre

Le périmètre est dimensionné sur la capacité réelle de l'équipe : l'estimation complète est de 40 à 62 jours-personne (JP), pour une capacité d'environ 30 JP (3 membres × ~10 jours effectifs). Le P0 est donc prioritaire, le P1 partiel et le P2 optionnel.

Contrainte	Description	Conséquence sur la conception
Temps	Cahier des charges le 24/10/2026 à 23:59 (UTC+1) ; réalisation en 7 itérations d'une semaine	Prototypes de faisabilité en semaine 1 ; une démo de bout en bout chaque semaine
Capacité	~30 JP disponibles contre 40–62 JP estimés	Couper CoAP, MCP et capteurs supplémentaires avant de toucher au flux principal ou à la sécurité
Budget	Budget étudiant : 1 Raspberry Pi, 1 BME280, 1 BH1750, câblage ; une VM Cloud (offre gratuite ou crédits étudiants)	Zones supplémentaires simulées dans Node-RED et étiquetées « simulated »
Matériel IA	Pas de GPU garanti ; le Raspberry Pi ne peut pas exécuter Llama	Endpoint Llama sur la VM, un PC de l'équipe ou un service hébergé ; réponse simulée pour les tests
Périmètre P0 (obligatoire)	1 capteur réel → passerelle → MQTT → backend → MongoDB → tableau de bord ; authentification ADMIN/CLIENT ; 1 alerte de seuil ; enregistrement d'objet et TD ; API REST déployée	Colonne vertébrale de l'architecture
Périmètre P1 (cible)	2e zone, WebSocket, graphiques d'historique, gestion des objets, IA en lecture seule avec RAG basique	Ajouté par itérations sans casser le P0
Périmètre P2 (bonus)	2e passerelle physique, découverte automatique complète, CoAP, MCP, buffer hors ligne, prédiction	Interfaces prévues mais non bloquantes
Hors périmètre	Action de conservation automatique, mesure certifiée, reconnaissance faciale, suivi des visiteurs, entraînement de modèle	Aucune commande d'actionneur critique dans l'architecture

7. Contraintes technologiques et de sécurité

7.1 Contraintes technologiques

Contrainte	Risque	Mesure retenue
Jakarta EE 11 et WildFly 41.0.1.Final imposés	Pas de fournisseur Jakarta NoSQL / Jakarta Data prêt pour MongoDB dans WildFly	Driver Java MongoDB officiel derrière une couche repository CDI ; prototype CRUD en semaine 1, versions figées
Node-RED et Node.js ne sont pas la même chose	Deux backends qui dupliquent utilisateurs, objets et alertes	Node-RED limité à l'intégration IoT ; pas de backend Node.js séparé
Un seul chemin d'ingestion	Une mesure traitée deux fois	Node-RED → endpoint Jakarta EE ; contrôle d'idempotence sur `measurementId`

Contrainte	Risque	Mesure retenue
GPIO du Raspberry Pi en 3,3 V, non tolérant au 5 V	Destruction de la carte	Capteurs I²C 3,3 V ; Arduino par USB série ; ADC pour les capteurs analogiques
Thing Description ≠ découverte	Objets non détectés automatiquement	Mécanisme d'annonce MQTT, validation et approbation ADMIN
Inférence LLM coûteuse	Latence, indisponibilité du modèle	IA derrière une interface de service ; prompts bornés ; mock pour les tests
Protocoles hétérogènes (I²C, MQTT, HTTPS, WSS, CoAP)	Incohérences d'identifiants et de schémas	Contrat commun : topics, schémas JSON et OpenAPI figés avant le développement en parallèle

7.2 Contraintes de sécurité

Domaine	Exigence
Transport	HTTPS/WSS pour navigateur et API ; MQTT sur TLS (port 8883) avec authentification des clients en déploiement.
Authentification	Connexion sécurisée, mots de passe hachés, jetons signés à durée limitée ; `iam` est une responsabilité logique intégrée à l'application Jakarta EE.
Autorisation	Contrôle côté serveur du rôle (ADMIN/CLIENT) et des zones sur chaque endpoint REST, abonnement WebSocket et outil IA ; masquer un bouton n'est pas une autorisation.
Moindre privilège	Le CLIENT lit et acquitte les alertes de ses zones ; les opérations ADMIN sont séparées et auditées.
Isolation réseau	MongoDB, l'endpoint Llama et l'administration du broker ne sont accessibles que sur le réseau interne ; seuls les ports 443 et 8883 sont ouverts.
Secrets	Variables d'environnement hors Git ; aucun jeton dans les logs ni dans les topics MQTT.
Confiance des objets	Toute annonce inconnue reste `pending` jusqu'à validation de l'identifiant, de la source et de la TD.
IA	Outils en lecture seule allowlistés, schémas validés, résultats filtrés par permissions avant d'atteindre le modèle ; aucune action irréversible ; le prompt ne peut jamais contourner l'autorisation.
Audit et vie privée	Journal des changements ADMIN, enregistrements d'objets, acquittements et appels d'outils ; données utilisateur minimales et durée de rétention définie.

8. Méthodologie XP et intégration des composants

Le projet suit eXtreme Programming : itérations d'une semaine, chacune terminée par une démo de bout en bout du flux capteur → tableau de bord. L'intégration est continue plutôt que regroupée en fin de projet.

Pratique XP	Application dans Smart Museum
User stories et critères d'acceptation	Une story par cas d'utilisation (UC-01 à UC-13), classée P0 / P1 / P2 dans le backlog
Petites livraisons	Tranche verticale dès la semaine 2 : une vraie mesure visible dans la PWA

Pratique XP	Application dans Smart Museum
Programmation en binôme	Chemins à risque : authentification, ingestion MQTT, validation TD, autorisation des outils IA
Tests d'abord	Tests unitaires, collection Postman des endpoints REST, tests de rôles, test du chemin capteur/MQTT
Intégration continue	Branche `main` toujours compilable ; branches courtes ; pull requests relues ; build automatique (GitHub Actions)
Propriété collective du code	Revue croisée entre les 3 membres ; en-têtes de fichiers normalisés (module, auteur, dates, version)
Refactoring	Autorisé sans changer le comportement convenu ; référence partagée mise à jour en même temps

8.1 Règles d'intégration

1. Contrat figé avant le travail en parallèle : topics MQTT, schéma de mesure, modèle de TD, API OpenAPI, codes d'erreur.
2. Identifiants stables partagés par tous les composants (`deviceId`, `zoneId`, `gatewayId`).
3. Capteurs simulés dans Node-RED pour que le backend et les PWA avancent sans attendre le matériel.
4. Déploiement sur la VM depuis GitHub par `git clone` / `git pull`, avec un environnement `test` avant la démo.
5. Tout changement d'architecture est tracé dans une issue GitHub (décision, raison, date, impact) et reporté dans la référence partagée.

Annexe A — Source PlantUML du déploiement

Code permettant de reproduire la figure 2 dans un outil UML (PlantUML, StarUML, draw.io).

```
@startuml SmartMuseum_Deploiement
node "Poste ou smartphone" <<device>> {
  node "Navigateur web" <<executionEnvironment>> as browser {
    artifact "PWA CLIENT"
    artifact "PWA ADMIN"
  }
}
node "Capteurs BME280 / BH1750" <<device>> as capteurs
node "Raspberry Pi 4/5" <<device>> {
  node "Raspberry Pi OS" <<executionEnvironment>> {
    artifact "gateway-agent" as agent
  }
}
node "VM Cloud IaaS (Linux)" <<device>> {
  node "Nginx :443" <<executionEnvironment>> as nginx {
    artifact "pwa-client"
    artifact "pwa-admin"
  }
  node "Mosquitto :8883" <<executionEnvironment>> as mqtt
  node "Node-RED :1880" <<executionEnvironment>> as nodered {
    artifact "flows.json"
  }
  node "WildFly 41.0.1.Final :8080" <<executionEnvironment>> as wildfly {
    artifact "smart-museum.war"
  }
  database "MongoDB :27017" as mongo
}
node "Serveur IA" <<device>> {
  node "Ollama" <<executionEnvironment>> as ollama {
    artifact "modèle Llama"
  }
}
node "GitHub" as github

capteurs --> agent : I²C
agent --> mqtt : MQTT/TLS 8883
mqtt --> nodered : MQTT 1883
nodered --> wildfly : HTTP POST /api/ingest
browser <--> nginx : HTTPS 443 / WSS
nginx --> wildfly : proxy 8080
wildfly <--> mongo : driver Java
wildfly <--> ollama : HTTP 11434
github ..> wildfly : git clone / pull
@enduml
```